
Tight analyses of first-order methods with error feedback

Daniel Berg Thomsen^{1,2*} Adrien Taylor¹ Aymeric Dieuleveut²

¹INRIA, D.I. École Normale Supérieure, PSL Research University, 75005 Paris, France

²CMAP, CNRS, École polytechnique, Institut Polytechnique de Paris, 91120 Palaiseau, France

Abstract

Communication between agents often constitutes a major computational bottleneck in distributed learning. One of the most common mitigation strategies is to compress the information exchanged, thereby reducing communication overhead. To counteract the degradation in convergence associated with compressed communication, error feedback schemes—most notably EF and EF²¹—were introduced. In this work, we provide a *tight analysis* of both of these methods. Specifically, we find the Lyapunov function that yields the best possible convergence rate for each method—with matching lower bounds. This principled approach yields sharp performance guarantees and enables a rigorous, apples-to-apples comparison between EF, EF²¹, and compressed gradient descent. Our analysis is carried out in the simplified single-agent setting, which allows for clean theoretical insights and fair comparison of the underlying mechanisms.

Remark: proof certificates

To consolidate and support our theoretical results, we complement each theoretical statement with analytical or numerical validation. Specifically, we provide certificates of correctness generated either with a *Computer Algebra System* (CAS), using a WolframScript, for symbolic verification, or using *Performance Estimation Problems* (PEP) for numerical validation. CAS enable verification of algebraic identities, while PEP annotations indicate numerical validation of complete statements. These certificates are highlighted in the paper using  and  markers, which are direct links to the corresponding Jupyter notebook or WolframScript in our public GitHub repository.^a

1 Introduction

Over the past decade, distributed optimization has become a cornerstone of large-scale machine learning. This shift is driven by major increases in the size of models and training data, as well as increasing societal concerns about data ownership and privacy. Ultimately, solutions in which training is distributed across a network of n agents, each retaining its own local data, under the coordination of a central server, have emerged as one of the most natural and efficient solutions to this problem [1, 2].

*Correspondence to daniel.berg-thomsen@inria.fr

^aWhile these certificates do not replace the mathematical proofs presented in the paper, they serve as an additional layer of transparency and error checking, analogous to unit tests in software development. This practice provides a reproducible, independently verifiable basis for our theoretical claims, thereby reducing the risk of oversights in complex derivations.

Formally, the goal is to solve the following minimization problem:

$$\min_{x \in \mathbb{R}^d} \left\{ f(x) := \frac{1}{n} \sum_{i=1}^n f_i(x) \right\}. \quad (1)$$

Classical methods such as distributed gradient descent and its stochastic variants achieve linear speedups in iteration complexity with respect to the number of agents. However, they often suffer from significant *communication overhead*, as gradients or model updates must be exchanged frequently over bandwidth-limited channels [3–5]. As the scale of models keep increasing, this communication bottleneck has been identified early on as a critical limitation, prompting the development of methods aimed at reducing communication costs. Two main strategies are favored: scarcely communicating with the central server, known as *local iterations* [see e.g. 1, 6]—and transmitting *compressed updates*, which aim to reduce the size of the exchanged information. Compression mechanisms can be applied to reduce communication either from agents to the server [3, 7–14] or from the server to the agents [15–23]. This paper focuses on methods using compression operators, which encompass a variety of strategies, including selecting only a fraction of the weights to be transmitted (e.g., the top K coordinates [8]) or communicating low-precision updates via quantization [7].

Algorithm 1 Compressed gradient descent (CGD)

- 1: **initialization:** $x_0 \in \mathbb{R}^d, \eta > 0$
 - 2: **for** $k = 0, 1, 2, \dots, N$ **do**
 - 3: Agent $i \in [n]$ compresses $\nabla f_i(x_k)$ and communicates $m_k^{(i)} := \mathcal{C}(\nabla f_i(x_k))$
 - 4: Server updates $x_{k+1} \leftarrow x_k - \eta \cdot \frac{1}{n} \sum_{i=1}^n m_k^{(i)}$
 - 5: **end for**
-

Formally, a compression operator is a possibly random mapping $\mathcal{C} : \mathcal{X} \rightarrow \mathcal{X}$, such that $\mathcal{C}(X)$ can be encoded (almost surely or on average) with a lower number of bits than X . The most natural algorithm leveraging communication compression with a centralized server is the *compressed gradient descent* algorithm (CGD), which is described in Algorithm 1. The main idea is to perform a distributed gradient step, with the compression operator $\mathcal{C}$ applied to the gradient of each agent before communication. Although this compression scheme reduces the communication cost, it comes at the expense of non-convergence in any practical setting [24].

To assess the general impact of compression schemes on the rate of convergence, one typically leverages the fact that these compressors all satisfy generic assumptions. These include unbiasedness, i.e., $\mathbb{E}[\mathcal{C}(x)] = x$ for any $x \in \mathcal{X}$, together with relatively bounded variance, which states that $\mathbb{E}[\|\mathcal{C}(x) - x\|^2] \leq \omega \|x\|^2$ for any $x \in \mathcal{X}$ [7, 11, 9, 25–27, 12, 28, 19, 20, 29–31], or *contractiveness* [32–34, 21, 24], defined as follows:

Assumption 1 (Contractive compression operator). *The compression operator $\mathcal{C}$ is a stochastic operator such that, for some $\epsilon \in [0, 1)$,*

$$\text{for all } x \in \mathbb{R}^d, \quad \mathbb{E}[\|x - \mathcal{C}(x)\|^2] \leq \epsilon \|x\|^2. \quad (2)$$

The standard way to improve CGD is to leverage the asymmetry of information: each agent has access to the exact gradient before compression and can therefore track the discrepancy between the exact gradient and the transmitted (compressed) message. This discrepancy can be stored and used as a correction term in subsequent iterations—a principle that lies at the heart of *error feedback techniques*. The most basic mechanism used is known as *classic error feedback* (EF), where each agent stores the difference between the true gradient and its compressed version locally, and incorporates this error into the next round of communication. This method, outlined in Algorithm 2, was first introduced in [3] and later analyzed in [32, 11, 10, 8, 35]. Notably, this method converges in many practical settings, effectively addressing the problem of non-convergence for CGD.

More recently, a variant of the classic error feedback mechanism, known as EF²¹ was introduced by Richtárik et al. [14], and is presented in Algorithm 3. Unlike classic error feedback, EF²¹ focuses on communicating a gradient estimate that is more robust to the variance observed in gradients received from *different* agents around the minimum of finite sum objectives. This method has since been extended in several directions [e.g. 23, 36–38].

Algorithm 2 Classic error feedback (EF)

```
1: initialization:  $x_0 \in \mathbb{R}^d, \eta > 0, e_0^{(i)} = 0$  for  $i = 1, \dots, n$   
2: for  $k = 0, 1, 2, \dots, N$  do  
3:   Agent  $i \in [n]$  compresses  $e_k^{(i)} + \eta \nabla f_i(x_k)$  and communicates  $m_k^{(i)} := \mathcal{C}(e_k^{(i)} + \eta \nabla f_i(x_k))$   
4:   Agent  $i \in [n]$  updates  $e_k^{(i)} \leftarrow e_k^{(i)} + \eta \nabla f_i(x_k) - \mathcal{C}(e_k^{(i)} + \eta \nabla f_i(x_k))$   
5:   Server updates  $x_{k+1} \leftarrow x_k - \frac{1}{n} \sum_{i=1}^n m_k^{(i)}$   
6: end for
```

Algorithm 3 Error Feedback 21 — EF²¹

```
1: initialization:  $x_0 \in \mathbb{R}^d$ ; step size  $\eta > 0$ ;  $d_0^{(i)} = \mathcal{C}(\nabla f_i(x_0))$  for  $i = 1, \dots, n$ ;  
2: for  $k = 0, 1, 2, \dots, N$  do  
3:   Server updates  $x_{k+1} \leftarrow x_k - \eta \cdot \frac{1}{n} \sum_{i=1}^n d_k^{(i)}$   
4:   Agent  $i \in [n]$  compresses  $\nabla f_i(x_{k+1}) - d_k^{(i)}$  and communicates  $m_k^{(i)} := \mathcal{C}(\nabla f_i(x_{k+1}) - d_k^{(i)})$   
5:   Agent  $i \in [n]$  updates  $d_{k+1}^{(i)} \leftarrow d_k^{(i)} + m_k^{(i)}$   
6: end for
```

Error feedback techniques are widely regarded as highly effective, and EF was described as “*compression for free*” as early as 2019 [10]. Despite that, and the abundant literature on the topic, the precise impact of error feedback techniques on performance remains difficult to assess. Comparison is complicated by the diversity of settings under which methods are analyzed: different function classes (smooth, convex, or nonconvex), a range of algorithmic enhancements (acceleration, adaptivity, variance reduction, etc.), and a variety of performance measures (different Lyapunov functions) [23, 39, 38, 37, 40–42]. While some works provide insightful counter-examples—e.g., Beznosikov et al. [24] show that classic error feedback effectively addresses the limitations of CGD in distributed settings—many others simply propose a Lyapunov function and establish an upper bound without demonstrating its tightness. As a result, claims about “compression for free” are often based on comparisons between potentially loose guarantees, which may not reliably reflect real algorithmic performance. The length and complexity of the proofs involved typically make it difficult to ensure the tightness of the results, and most proofs are constructed in an ad hoc manner. Consequently, it is difficult to determine which methods are actually worst-case optimal based on upper bounds whose tightness is not always assured.

As a result, even remarkably simple questions remain only partially answered:

What is the optimal convergence rate that each method can attain?
Given an optimization setting, what method should we choose?
How should each method be optimally tuned?

Our goal is to provide definitive answers to parts of these questions. In this paper, we take a complementary perspective to the existing literature and offer a tight, principled comparison of the three methods. Specifically, we derive their optimal tuning, identify an optimal Lyapunov function for each method, and compute the *exact* optimal convergence rate for *any* Lyapunov function within our class of candidate Lyapunov functions.

To make this comparison sharp and transparent, we adopt a deliberately simple yet representative setup: we consider smooth and strongly convex functions in the single-agent setting ($n = 1$). While simple, this regime is widely recognized as a crucial stepping stone—not only for building intuition, but also as its own theoretical contribution [e.g., 32, 10]. The single-agent setting is also of independent interest: in the field of *sparsity-aware* neural-network training, sparse-update methods have been shown to correspond to error feedback [43]. In this context, *tightness* means that we identify the best possible Lyapunov function within a given class *and* compute the exact worst-case convergence rate over the class of problems considered.

Our methodology draws on the *performance estimation* framework [44, 45], which enables the numerical derivation of exact convergence rates for a wide range of first-order methods. In particular,

recent advances [46, 47] demonstrate how to automatically search for optimal Lyapunov functions. While these approaches are primarily numerical, we build upon insights from their underlying proof structures [48] to derive new analytical results.

Contributions. This work makes the following contributions:

1. From a methodological perspective, this paper shows how to apply the performance estimation framework to algorithms from the *federated learning* literature that incorporate compression schemes. By leveraging this methodology—both analytically and numerically—this work paves the way for a more precise and reliable understanding of federated and distributed learning methods.
2. This work provides a *tight* analysis of EF and EF²¹, and compare them with compressed gradient descent in the single-agent setting, on L -smooth, μ -strongly convex functions. In particular, we give an analytical formula of the best possible contraction rate, by analyzing an optimal Lyapunov function within a class of candidate Lyapunov functions defined in Definition 1. Furthermore, we provide the optimal tuning for the step size in both those algorithms.
3. We demonstrate that those rates are achieved, proving that the analysis is tight.
4. We conclude that the complexities of EF and EF²¹ are perfectly identical in this particular setting. Moreover, CGD outperforms both methods—both in terms of the range of settings where it converges and in terms of the optimal convergence rate achieved.
5. Finally, we contribute to the process of deriving *simple* Lyapunov functions for first-order methods, and extend known results for fixed-step methods to the setting of methods using compression.

Paper outline. The rest of the paper is organized as follows. In Section 2, we provide background on the relevant existing results for CGD, EF, and EF²¹. We also provide the necessary background on the techniques from the performance estimation literature needed to outline the methodology we use, as well as the definition of the classes of Lyapunov functions used. Section 3 presents the main contribution of the paper: tight convergence guarantees for CGD, EF, and EF²¹, along with matching lower bounds. Section 4 details the methodology we use to derive the results, and provides references to the formal results required to justify this approach. It also contains a number of numerical results that illustrate the equivalence between EF and EF²¹, and performance characteristics of the three methods. Section 5 summarizes the results of the paper and provides a discussion of the results in relation to the points brought up in the introductory section.

Notations: We denote $\mathbb{S}^\ell$ the symmetric matrices, and denote $\mathbb{S}_+^\ell$ the set of positive semi definite matrices. For any two matrices $A \in \mathbb{S}^\ell$ and $B \in \mathbb{S}^d$, we denote $A \otimes B$ the Kronecker product.

2 Background

In this section, we briefly overview relevant existing results from the field of distributed optimization, the necessary background on the performance estimation framework, provide the rest of the assumptions we will need, and specify the notion of Lyapunov functions used in this paper.

2.1 Theoretical results on CGD, EF, EF²¹

In the single agent case, we leverage the equivalence between compressed gradient descent (CGD), under Assumption 1 and the *inexact gradient method* with relatively bounded gradients. CGD corresponds to the particular case of Algorithm 1 with $n = 1$, and relatively bounded gradients means that for any x , the oracle queried at point x outputs a value g such that $\|g - \nabla f(x)\|^2 \leq \epsilon \|\nabla f(x)\|^2$. Various notions of gradient approximation have been studied [49–51], and a tight analysis for relatively bounded gradients was given in [52]. Specifically, authors have shown that the inexact gradient method then enjoys tight convergence guarantees for any step size $\eta > 0$, with respect to the functional residual, Euclidean norm distance to the solution and gradient norm. However, CGD is known to diverge when applied using stochastic gradient oracles, and to non-smooth functions [10]. Interestingly, it is also known to diverge in the multi-worker setting [24]. Studying CGD is important in its own right, because when the compression operator is chosen as the sign function, and the algorithm is applied in the stochastic setting (i.e., signSGD), there is a connection to Adam both in the convex [53], and non-convex setting [54].

Convergence rates for EF have been established with strongly convex [32], quasi-convex and non-convex [10, 35] functions, and using stochastic gradients. Richtárik et al. [14] study EF²¹ in the multi-worker setting, with (potentially) randomized compression operators. They establish a $\mathcal{O}(k^{-1})$ convergence rate on Lipschitz smooth functions, and a linear rate under the additional assumption that the functions satisfy the Łojasiewicz inequality. These results are obtained using a Lyapunov function; however, without tightness guarantees—neither for the choice of Lyapunov function, nor for the convergence rate itself. Extensions of EF²¹ have been proposed, including adaptations to stochastic gradients [23], and the introduction of a momentum term to improve sample complexity in the stochastic setting [36].

2.2 Performance estimation

Performance estimation tools [55, 56, 45] enable to obtain tight (i.e., exact worst-case) numerical guarantees on convergence rates for various choices of Lyapunov functions. To do so, the estimation of the worst-case rate is formulated as a semidefinite program (SDP), which is then solved numerically using standard solvers such as MOSEK [57]. The resulting numerical values approximate the exact *worst-case* rate of an algorithm over a class of functions, and should not be confused with quantities that depend on specific data, initial points, or problem instances.

This framework has been made accessible through software packages in both Python [58] and Matlab [59], enabling researchers to easily apply these tools. Advanced performance estimation techniques based on the dual formulation of the aforementioned SDP have been developed within this framework to construct optimal Lyapunov functions for first-order methods [46, 47, 60]. Particularly relevant is the approach of [46], which formulates the search for *quadratic* Lyapunov functions as a feasibility problem with a candidate contraction rate. By performing bisection on this rate, the method identifies the smallest contraction rate for which a valid Lyapunov function exists.

Another relevant line of work we leverage to discover the analytical form of the Lyapunov functions lies in the field of *symbolic regression*, which aims to solve supervised learning tasks over the space of simple analytic expressions. Recent advances use genetic programming to search this space, and software packages have been developed in both Python and Julia [61].

2.3 Definitions & Notation

We have already introduced the notion of a contractive compression operator in Assumption 1. To position our contribution within the broader literature we now specify that our analysis is restricted to the setting of smooth, strongly convex functions:

Assumption 2. *The function f is L -smooth, i.e., for all $x, y \in \mathbb{R}^d$, we have*

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|^2.$$

Assumption 3. *The function f is μ -strongly convex, i.e., for all $x, y \in \mathbb{R}^d$, we have*

$$f(y) \geq f(x) + \langle \nabla f(x), y - x \rangle + \frac{\mu}{2} \|y - x\|^2.$$

We will use the notation $\mathcal{F}_{\mu,L}$ to denote the set of smooth, strongly convex functions with parameters μ and L . We will denote $\kappa := \frac{L}{\mu}$ the condition number. For any objective function $f \in \mathcal{F}_{\mu,L}$, we denote $x_\star := \arg \min_{x \in \mathbb{R}^d} f$ its minimizer, and $f_\star := \min_{x \in \mathbb{R}^d} f(x)$ its minimum value.

Lyapunov functions. We now formally define the class of Lyapunov functions under consideration.

We formally denote $\mathcal{M} : \mathbb{R}^{\ell \times d} \times \mathbb{R}^d \times \mathcal{F} \rightarrow \mathbb{R}^{\ell \times d} \times \mathbb{R}^d$ a first-order method acting on a set of functions $\mathcal{F}$, for an integer $\ell \in \mathbb{N}$. Such a method, given a function $f \in \mathcal{F}$, is applied to an initial *state* $\xi_0 \in \mathbb{R}^{\ell \times d}$ and iterate $x_0 \in \mathbb{R}^d$, and generates a sequence $\{\xi_k\}_{k \geq 0}$ of states, and a sequence $\{x_k\}_{k \geq 0}$ of iterations. The *states* represent information summarizing the current point in the optimization trajectory that the algorithms may depend on beyond the current iterate—for example, error-related quantities in error feedback algorithms. The integer ℓ is thus typically small, from 0 to 3 in general. The specific states used in this paper are all specified in Subsection 3.1.

Definition 1 (Candidate Lyapunov function). *A function $\mathcal{V} : \mathbb{R}^{\ell \times d} \times \mathbb{R}^d \rightarrow \mathbb{R}$ is called a candidate Lyapunov function for f if it satisfies the following conditions:*

1. (Non-negativity) $\mathcal{V}(\xi, x; f) \geq 0$, for any $\xi \in \mathbb{R}^{\ell \times d}$, $x \in \mathbb{R}^d$,
2. (Zero at fixed-point) $\mathcal{V}(\xi, x; f) = 0$ if and only if $x = x_*$ and $\xi = \xi_*$ for a unique $\xi_* \in \mathbb{R}^{\ell \times d}$.
3. (Meaningfully lower bounded) there exists a positive semidefinite matrix $A \in \mathbb{S}_+^\ell$ and a scalar $a \geq 0$ such that $\mathcal{V}(\xi, x; f) \geq (\xi - \xi_*)^\top (A \otimes I_d) (\xi - \xi_*) + a(f(x) - f_*)$ and $\text{Tr}(A) + a = 1$.

The lower bound in item 3 of our definition requires some justification: it ensures that the Lyapunov function provides control over meaningful quantities in optimization, such as the distance to the fixed point, gradient norm, algorithm-dependent quantities and the functional residual.

The class of candidate Lyapunov functions is thus given by

$$\mathbb{V}_\ell = \{(P, p) \in \mathbb{S}_+^\ell \times \mathbb{R}^+ : \text{Tr}(P) + p = 1\}. \quad (3)$$

For any $(P, p) \in \mathbb{V}_\ell$ we denote $\mathcal{V}_{(P, p)}$ the Lyapunov functions of the form:

$$\mathcal{V}_{(P, p)}(\xi, x; f) = (\xi - \xi_*)^\top (P \otimes I_d) (\xi - \xi_*) + p(f(x) - f_*). \quad (4)$$

We seek candidate Lyapunov functions $\mathcal{V} : \mathbb{R}^{\ell \times d} \times \mathbb{R}^d \times \mathcal{F} \rightarrow \mathbb{R}$ that satisfy the recurrence

$$\mathcal{V}(\xi_{k+1}, x_{k+1}; f) \leq \rho \cdot \mathcal{V}(\xi_k, x_k; f), \quad (5)$$

for some constant $\rho < 1$ and for all $k \geq 0$, uniformly over $\mathcal{F}$. Finding the *optimal* Lyapunov function within a parameterized class, for a method $\mathcal{M}$, then amounts to solving the following problem:

$$\rho^*(\mathcal{M}) := \min_{(P, p) \in \mathbb{V}_\ell} \left\{ \max_{\substack{f \in \mathcal{F}_{\mu, L}, \\ (\xi_0, x_0) \in \mathbb{R}^{\ell \times d} \times \mathbb{R}^d}} \frac{\mathcal{V}_{(P, p)}(\xi_1, x_1; f)}{\mathcal{V}_{(P, p)}(\xi_0, x_0; f)} : (\xi_1, x_1) = \mathcal{M}(\xi_0, x_0; f) \right\}. \quad (6)$$

Note that we set $k = 0$ in order to have a guarantee that is valid for all $x \in \mathbb{R}^d$.

3 Main results

In this section, we provide answers to the questions stated in the introduction for our setting. We begin by showing some numerical results on the performance of each method, and then provide the precise statements of all of our theoretical results.

3.1 Numerical performance of all methods

In order to compare EF and EF²¹ with the performance of CGD, we first need to specify the state-variables under consideration when analyzing each method. Those are given by:

$$\xi_k^{\text{CGD}} = \begin{bmatrix} x_k \\ \nabla f(x_k) \\ \mathcal{C}(\eta \nabla f(x_k)) \end{bmatrix}, \quad \xi_k^{\text{EF}} = \begin{bmatrix} x_k \\ \nabla f(x_k) \\ \mathcal{C}(e_k + \eta \nabla f(x_k)) \end{bmatrix}, \quad \xi_k^{\text{EF}^{21}} = \begin{bmatrix} x_k \\ \nabla f(x_k) \\ d_k \end{bmatrix}, \quad (7)$$

where all variables are defined as in Algorithm 1, Algorithm 2 and Algorithm 3, respectively. Note that all numerical results of this article are using *deterministic* compression operators.

We are now ready to present the numerical results on the performance of each method. Figure 1 shows contour plots of the contraction factor for each method. That is, for a fine grid of both the step size η in Algorithms 1 to 3, and the parameter ϵ in Assumption 1, we numerically compute the value of the best possible worst-case contraction rate in terms of our class of candidate Lyapunov functions, over $\mathcal{F}_{\mu, L}$, as given by (6). A darker blue point indicates a stronger contraction $\rho^*(\mathcal{M})$ (i.e., a better rate). A red point indicates that the method is non-convergent for that choice of (ϵ, η) . We observe that, although the results are purely numerical at this stage, they indicate that EF and EF²¹ exhibit identical performance in our setting. This numerical equivalence is supported by Table 1: the maximum absolute difference between contraction factors for EF and EF²¹ is on the order of 10^{-5} to 10^{-7} . This first fact is a very surprising observation. Indeed while EF and EF²¹ are known to be identical in the very specific case of using a deterministic positively homogeneous and additive

	$\kappa = 2$	$\kappa = 4$	$\kappa = 10$		$\kappa = 2$	$\kappa = 4$	$\kappa = 10$
Absolute error	4.34e-07	6.02e-07	1.21e-06	Absolute error	1.14e-06	3.90e-07	5.11e-06

Table 1: Maximum absolute difference of contraction factor for EF and EF^{21} , computed over grid of $\epsilon \in [0.01, 0.99]$ and $\eta \in [0.01, \frac{2}{L+\mu}]$ for $L = 1$, and varying μ .

Table 2: Maximum absolute difference of contraction factor for CGD when allowing any combination of terms in our Lyapunov function, compared to the contraction achieved by the functional residual. Same grid as Table 1.

compression operator [14, see Section 4.2], EF and EF^{21} remain grounded in fundamentally different motivations at first sight: on the one hand, EF accounts for the errors introduced by the compression step, while on the other hand, EF^{21} subtracts a control variate from the gradient prior to compression. Proving that the best possible convergence rate they can obtain, for any tuning ϵ, η , is similar (but achieved for a *different* Lyapunov function) was, to the best of our knowledge, never established in the literature. It thus constitutes a significant step towards better understanding their connections.

A second observation can be made from those plots: the region of non-convergence is by far larger for EF and EF^{21} than for CGD. In particular, there exist multiple tunings, for which incorporating any of the two types of error feedback, actually *prevent* convergence.

While given for a single (μ, L) in Figure 1 and Table 1, similar results hold for all values of (μ, L) that were tried numerically, and several examples are given in Appendix D, along with details of the numerical experiments, including the computation of regions of non-convergence.

Furthermore, we *tune* each algorithm by picking the optimal step size for each method. We compute the rate $\inf_{\eta} \rho^*(\mathcal{M}_{\eta})$, for $\mathcal{M} \in \{\text{CGD}, \text{EF}, \text{EF}^{21}\}$, where $\mathcal{M}_{\eta}$ corresponds to the method with step size η . Results are shown in Appendix B for three values of κ , namely 2, 4, and 10. In each setup, for every level of compression, CGD achieves a rate which is strictly better than EF and EF^{21} . In Appendix B.1, we also provide a symbolic certificate of this fact. These results challenge the prevailing intuition that error feedback ensures convergence comparable to that of uncompressed methods, and even demonstrate that in the single agent and deterministic gradient regime, error feedback is actually always detrimental to convergence.

Finally, we note that the functional residual constitutes an optimal Lyapunov function for CGD, as shown in Table 2. This result is not particularly surprising, given that a tight analysis of the functional residual with the optimal step size was previously established for *inexact gradient descent* by De Klerk et al. [52]. That work also demonstrated the tightness of this rate². For EF^{21} , Richtárik

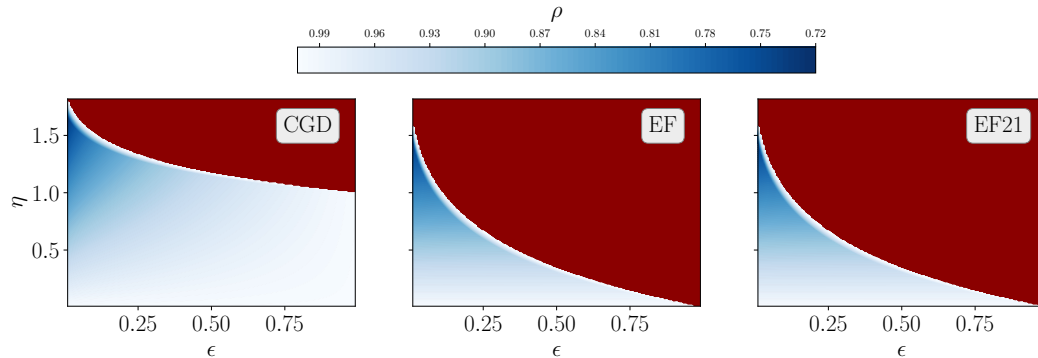


Figure 1: Single row of contour plots showing performance of CGD, EF, and EF^{21} as a function of step size η and compression parameter ϵ , with regions of non-convergence marked in red. The regions of non-convergence were computed using PEPit by finding cycles of length 2.

et al. [14] proposed another Lyapunov function than the one used here. In Appendix D.5, Figure 12 we compare the complexities of an optimally tuned version of their Lyapunov function with the class-optimal Lyapunov function numerically. Further comparison of rates can be found in Appendix B. In Appendix B.2, we prove that our rate is strictly faster than the the aforementioned rate for EF^{21} .

In the next two sections, we provide analytical results on EF and EF^{21} , respectively.

3.2 Exact convergence rate and optimal tuning for EF

We begin by stating the main result of this section, which is a tight rate of convergence for EF.

Theorem 1. CAS PEP

Consider running Algorithm 2, i.e., EF, with a compression operator $\mathcal{C}$ satisfying Assumption 1 for some $\epsilon \in [0, 1]$ on any function satisfying Assumptions 2, and 3. Let the step size be given by

$$\eta^* = \left(\frac{2}{L + \mu} \right) \cdot \left(\frac{1 - \sqrt{\epsilon}}{1 + \sqrt{\epsilon}} \right). \quad (8)$$

Then, we have that

$$\rho^*(\text{EF}_{\eta^*}) = \sqrt{\epsilon} + \frac{1}{4}(1 + \sqrt{\epsilon})(L - \mu)\lambda, \quad (9)$$

where

$$\lambda := \frac{\eta^*}{L + \mu} \left[(1 - \sqrt{\epsilon})(L - \mu) + (1 + \sqrt{\epsilon})\sqrt{(L - \mu)^2 + 16L\mu\frac{\sqrt{\epsilon}}{(1 + \sqrt{\epsilon})^2}} \right]. \quad (10)$$

A Lyapunov function achieving the rate in (9), with ξ^{EF} defined in (7), is given by

$$\mathcal{V}(\xi^{\text{EF}}, x; f) := \|x - x_\star\|^2 - 2(x - x_\star)^\top e + \left(1 + \frac{1}{\sqrt{\epsilon}}\right) \cdot \|e\|^2 = \|x - x_\star - e\|^2 + \frac{1}{\sqrt{\epsilon}}\|e\|^2, \quad (11)$$

Finally, the step size in (8) is worst-case optimal for EF: $\forall \eta \geq 0$, we have $\rho^*(\text{EF}_\eta) \geq \rho^*(\text{EF}_{\eta^*})$.

Importantly, (9) shows that the rate is tight; that is, there exist $f \in \mathcal{F}_{\mu, L}$ and $(P, p) \in \mathbb{V}_\ell$ for which the rate is exactly achieved. Since the lower bound also applies to any other performance measure in our state space, our performance measure is optimal. This is formally demonstrated in the proof provided in Appendix C.1. The proof was written using *deterministic* compressors, but the same guarantees hold under expectation with stochastic compressors as is explained in Appendix C.3.

For completeness—and to support both our theoretical results and the tightness of the numerical results obtained through performance estimation—we provide additional figures comparing the empirically observed optimal step sizes for worst-case instances with our theoretical step size η^* , across different values of ϵ and μ/L . These results are shown in Figure 11, located in Appendix D.4. The numerical and analytical values match up to numerical accuracy.

3.3 Exact convergence rate and optimal tuning for EF^{21}

We now state our main result on the EF^{21} algorithm, which is also tight.

Theorem 2. CAS PEP

Consider running Algorithm 3 with a compression operator satisfying Assumption 1 for some $\epsilon \in [0, 1]$ on any function satisfying Assumptions 2, and 3. Let the step size be given by η^* in (8). Then,

$$\rho^*(\text{EF}^{21}_{\eta^*}) = \rho^*(\text{EF}_{\eta^*}). \quad (12)$$

A Lyapunov function achieving the rate in (12) is given by

$$\mathcal{V}(\xi^{\text{EF}^{21}}, x; f) := (1 + \sqrt{\epsilon}) \cdot \|g\|^2 - 2g^\top d + \|d\|^2 = \|g - d\|^2 + \sqrt{\epsilon} \cdot \|g\|^2. \quad (13)$$

Finally, the step size η^* is worst-case optimal for this algorithm.

Remark 1. The explicit rate for EF and EF^{21} can be written as

$$\rho = \sqrt{\epsilon} + \left(\frac{1 - \sqrt{\epsilon}}{2} \right) \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 \left[1 - \sqrt{\epsilon} + \sqrt{(1 + \sqrt{\epsilon})^2 + \sqrt{\epsilon}16\frac{\kappa}{(\kappa - 1)^2}} \right], \quad (14)$$

where $\kappa = L/\mu$. A detailed comparison between this and the existing rate for EF^{21} under the Łojasiewicz inequality [14] is provided in Appendix B.

The proof of this theorem is given in Appendix C.2 and assumes *deterministic* compressors, but the same guarantees hold under expectation with stochastic compressors, as is explained in Appendix C.3.

This second theoretical result analytically confirms the surprising numerical observation from Subsection 3.1, illustrated in Figure 1: EF and EF²¹ have the exact same optimal guarantee. Furthermore, the optimal step size is also the same, and can also be argued to be worst-case optimal in our setting using the same arguments as in Section 3.2. However, the optimal Lyapunov function is very different.

3.4 Tightness over multiple iterations and choice of state variables

A natural question arising from the above analysis concerns the tightness of the Lyapunov functions provided in Theorems 1 and 2 *over multiple steps*. Specifically, for $K \geq 2$, we investigate whether the convergence rate of a method $\mathcal{M} \in \text{EF}, \text{EF}^{21}, \text{CGD}$ can be improved by analyzing $\mathcal{M}^K$, the method run over K iterations, defined as follows:

$$\rho_K^*(\mathcal{M}) := \left(\min_{(P,p) \in \mathbb{V}_\ell} \left\{ \max_{\substack{f \in \mathcal{F}_{\mu,L}, \\ (\xi_0, x_0) \in \mathbb{R}^{\ell \times d} \times \mathbb{R}^d}} \frac{\mathcal{V}_{(P,p)}(\xi_K, x_K; f)}{\mathcal{V}_{(P,p)}(\xi_0, x_0; f)} : (\xi_K, x_K) = \mathcal{M}^K(\xi_0, x_0; f) \right\} \right)^{1/K}.$$

We provide a numerical answer to that question in Appendix D.2 for all algorithms discussed in the paper, showing that the analysis of the single-state Lyapunov functions used in this work are tight even if we consider multiple iterations. To that end, we plot the worst-case contractions for multiple iterations computed using PEPit [58], on the optimal Lyapunov functions given by (11) and (13).

Lyapunov Tightness. To prove that the Lyapunov functions we use in Theorems 1 and 2 are tight, one has to show that the rate of convergence for any other candidate Lyapunov function from our class is lower bounded by the rate we obtain. We consider a quadratic function, used in the latter sections of our proofs to prove tightness: asymptotic expressions for all the state-variables are the same, up to an iteration-independent constant. Consequently, when computing ratios between any set of Lyapunov functions, these constants cancel out. The only thing remaining is the term that actually depends the iteration count, which is exactly equal to the rate given in Theorems 1 and 2.

4 Methodology

We now present the methodology used to obtain the results in Section 3. The approach builds on the framework developed by [46]. We extend it to cover methods using *deterministic* compression operators under Assumption 1 in Appendix A. Obtaining the proofs of Theorems 1 and 2 required a combination of advanced performance estimation techniques (finding optimal Lyapunov functions), several tricks, as well as symbolic computation and symbolic regression frameworks.

To solve problem (6), we begin by addressing the inner maximization problem for a fixed contraction factor ρ . This amounts to checking the feasibility of a semidefinite program, detailed in Appendix A. We then apply bisection on ρ to identify the smallest admissible contraction factor. However, the Lyapunov function given is rarely unique, and most solutions obtained numerically vary significantly with problem parameters such as the compression factor ϵ . To address this, we use rank minimization heuristics—specifically, the logdet heuristic [62]. This enables one to obtain a unique set of structurally simpler, low-rank Lyapunov functions. Finally, we proceed by eliminating redundant coefficients in the matrix P and the scalar p , to arrive at the concise forms presented in Section 3. At the end of this process, the Lyapunov function coefficients were found to be mutually dependent, reducing the problem to identifying a closed-form expression for any one of them. To estimate such a coefficient, we applied symbolic regression using the PySR Python package [61]. This approach proved highly effective at finding simple yet *optimal* Lyapunov functions. To arrive at simple and readable proofs, we leverage the computer algebra system of *Mathematica* [63].

We wish to emphasize that the combined use of log-det heuristics, symbolic regression, and a computer algebra system turned out to be highly effective at solving this problem, and we believe it has broad applicability to other problems in machine learning.

5 Conclusion

In this paper, we provided tight analyses of EF and EF²¹ using Lyapunov functions, with guarantees on both the Lyapunov functions themselves and the convergence rates achieved. Notably, both algorithms exhibit the same convergence rate in our setting, and through a remark made in the discussion below, this gives us tight rates for any of the candidate Lyapunov functions we considered in our class. We also observed that their performance is strictly worse than that of compressed gradient descent—an outcome that, we believe, challenges the intuition of many in the field.

Our analysis is confined to the single-agent setting, both as an interesting problem on its own, and as a source of intuition for the multi-agent case. CGD cannot serve as a baseline in the multi-agent setting as it fails to converge with more than one agent [24]. In contrast, EF²¹ was specifically designed to *improve* convergence over EF in the multi-agent setting. Yet, its convergence rate in the single-agent case matches exactly that of EF. The findings of this paper raise two compelling questions:

- *Does the performance of EF and EF²¹ differ in the multi-agent setting?*
- *Are there more effective error compensation mechanisms yet to be discovered?*

We leave these questions for future work and conclude by emphasizing that the methodology used is likely applicable to a broad range of problems in optimization for machine learning. We look forward to seeing it extended and applied in future research.

Acknowledgments and Disclosure of Funding

We would like to thank Si Yi Meng, Jean-Baptiste Fest, Abel Douzal, and Lucas Versini for providing helpful feedback on an early draft of this paper. D. Berg Thomsen and A. Taylor are supported by the European Union (ERC grant CASPER 101162889). The work of A. Dieuleveut is partly supported by ANR-19-CHIA-0002-01/chaire SCAI, and Hi!Paris FLAG project, PEPR Redeem. The French government also partly funded this work under the management of Agence Nationale de la Recherche as part of the “France 2030” program, references ANR-23-IACL-0008 “PR[AI]RIE-PSAI”, ANR-23-PEIA-005 (REDEEM project) and ANR-23-IACL-0005.

References

- [1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, April 2017.
- [2] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Keith Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D'Oliveira, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badi Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaid Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrède Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Mariana Raykova, Hang Qi, Daniel Ramage, Ramesh Raskar, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. Advances and Open Problems in Federated Learning. *arXiv:1912.04977 [cs, stat]*, December 2019.
- [3] Frank Seide, Hao Fu, Jasha Droppo, Gang Li, and Dong Yu. 1-Bit Stochastic Gradient Descent and its Application to Data-Parallel Distributed Training of Speech DNNs. In *Annual Conference of the International Speech Communication Association*, 2014.
- [4] Trishul Chilimbi, Yutaka Suzue, Johnson Apacible, and Karthik Kalyanaraman. Project adam: Building an efficient and scalable deep learning training system. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, 2014.
- [5] Nikko Strom. Scalable distributed DNN training using commodity GPU cloud computing. In *Annual Conference of the International Speech Communication Association*, 2015.
- [6] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. Scaffold: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [7] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [8] Dan Alistarh, Torsten Hoefler, Mikael Johansson, Nikola Konstantinov, Sarit Khirirat, and Cedric Renggli. The Convergence of Sparsified Gradient Methods. *Advances in Neural Information Processing Systems (NeurIPS)*, 31, 2018.
- [9] K. Mishchenko, E. Gorbunov, M. Takáč, and P. Richtárik and. Distributed learning with compressed gradient differences. *Optimization Methods and Software*, 2024.
- [10] Sai Praneeth Karimireddy, Quentin Rebjock, Sebastian Stich, and Martin Jaggi. Error Feedback Fixes SignSGD and other Gradient Compression Schemes. In *International Conference on Machine Learning (ICML)*, 2019.
- [11] Jiaxiang Wu, Weidong Huang, Junzhou Huang, and Tong Zhang. Error Compensated Quantized SGD and its Applications to Large-scale Distributed Optimization. In *International Conference on Machine Learning (ICML)*, 2018.
- [12] Samuel Horvath, Chen-Yu Ho, Ludovik Horvath, Atal Narayan Sahu, Marco Canini, and Peter Richtárik. Natural Compression for Distributed Deep Learning. In *Mathematical and Scientific Machine Learning*, 2022.
- [13] Zhize Li, Dmitry Kovalev, Xun Qian, and Peter Richtarik. Acceleration for Compressed Gradient Descent in Distributed and Federated Optimization. In *International Conference on Machine Learning (ICML)*, 2020.
- [14] Peter Richtárik, Igor Sokolov, and Ilyas Fatkhullin. EF21: A New, Simpler, Theoretically Better, and Practically Faster Error Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

- [15] Ibrahim El Khalil Harrane, Rémi Flamary, and Cédric Richard. On reducing the communication cost of the diffusion lms algorithm. *IEEE Transactions on Signal and Information Processing over Networks*, 5(1):100–112, 2018.
- [16] Hanlin Tang, Chen Yu, Xiangru Lian, Tong Zhang, and Ji Liu. DoubleSqueeze: Parallel Stochastic Gradient Descent with Double-pass Error-Compensated Compression. In *International Conference on Machine Learning (ICML)*, 2019.
- [17] Xiaorui Liu, Yao Li, Jiliang Tang, and Ming Yan. A Double Residual Compression Algorithm for Efficient Distributed Learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2020.
- [18] Shuai Zheng, Ziyue Huang, and James Kwok. Communication-Efficient Distributed Blockwise Momentum SGD with Error-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [19] Constantin Philippenko and Aymeric Dieuleveut. Bidirectional compression in heterogeneous settings for distributed or federated learning with partial participation: tight convergence guarantees. *arXiv:2006.14591 [cs, stat]*, 2020.
- [20] Constantin Philippenko and Aymeric Dieuleveut. Preserved central model for faster bidirectional compression in distributed settings. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [21] Eduard Gorbunov, Dmitry Kovalev, Dmitry Makarenko, and Peter Richtarik. Linearly Converging Error Compensated SGD. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [22] Felix Sattler, Simon Wiedemann, Klaus-Robert Müller, and Wojciech Samek. Robust and Communication-Efficient Federated Learning From Non-i.i.d. Data. *IEEE Transactions on Neural Networks and Learning Systems*, pages 3400–3413, 2019. ISSN 2162-2388. doi: 10.1109/TNNLS.2019.2944481.
- [23] Ilyas Fatkhullin, Igor Sokolov, Eduard Gorbunov, Zhize Li, and Peter Richtárik. EF21 with Bells & Whistles: Practical Algorithmic Extensions of Modern Error Feedback, October 2021. *arXiv:2110.03294 [cs, math]*.
- [24] Aleksandr Beznosikov, Samuel Horváth, Peter Richtárik, and Mher Safaryan. On Biased Compression for Distributed Learning. *Journal of Machine Learning Research*, 24(276):1–50, 2023.
- [25] Sélim Chraïbi, Ahmed Khaled, Dmitry Kovalev, Peter Richtárik, Adil Salim, and Martin Takáč. Distributed Fixed Point Methods with Compressed Iterates. *arXiv:1912.09925 [cs, math]*, December 2019.
- [26] Eduard Gorbunov, Filip Hanzely, and Peter Richtárik. A Unified Theory of SGD: Variance Reduction, Sampling, Quantization and Coordinate Descent. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2020.
- [27] Amirhossein Reisizadeh, Aryan Mokhtari, Hamed Hassani, Ali Jadbabaie, and Ramtin Pedarsani. FedPAQ: A Communication-Efficient Federated Learning Method with Periodic Averaging and Quantization. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2020.
- [28] Dmitry Kovalev, Elnur Gasanov, Alexander Gasnikov, and Peter Richtarik. Lower Bounds and Optimal Algorithms for Smooth and Strongly Convex Decentralized Optimization Over Time-Varying Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [29] Farzin Haddadpour, Mohammad Mahdi Kamani, Aryan Mokhtari, and Mehrdad Mahdavi. Federated Learning with Compression: Unified Analysis and Sharp Guarantees. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2021.

- [30] Zhize Li and Peter Richtárik. CANITA: Faster Rates for Distributed Convex Optimization with Communication Compression. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [31] Sarit Khirirat, Hamid Reza Feyzmahdavian, and Mikael Johansson. Distributed learning with compressed gradients. *arXiv:1806.06573 [cs, math]*, June 2018.
- [32] Sebastian U Stich, Jean-Baptiste Cordonnier, and Martin Jaggi. Sparsified SGD with Memory. In *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.
- [33] Nikita Ivkin, Daniel Rothschild, Enayat Ullah, Vladimir Braverman, Ion Stoica, and Raman Arora. Communication-efficient Distributed SGD with Sketching. *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [34] Anastasia Koloskova, Sebastian Stich, and Martin Jaggi. Decentralized Stochastic Optimization and Gossip Algorithms with Compressed Communication. In *International Conference on Machine Learning (ICML)*, 2019.
- [35] Sebastian U Stich and Sai Praneeth Karimireddy. The error-feedback framework: Better rates for SGD with delayed gradients and compressed updates. *Journal of Machine Learning Research*, 21:1–36, 2020.
- [36] Ilyas Fatkhullin, Alexander Tyurin, and Peter Richtárik. Momentum provably improves error feedback! *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [37] Kaja Gruntkowska, Alexander Tyurin, and Peter Richtárik. EF21-P and Friends: Improved Theoretical Communication Complexity for Distributed Optimization with Bidirectional Compression. In *International Conference on Machine Learning (ICML)*, 2023.
- [38] Dmitry Makarenko, Elnur Gasanov, Rustem Islamov, Abdurakhmon Sadiev, and Peter Richtárik. Adaptive Compression for Communication-Efficient Distributed Training. *arXiv:2211.00188 [cs]*, October 2022.
- [39] Peter Richtarik, Igor Sokolov, Elnur Gasanov, Ilyas Fatkhullin, Zhize Li, and Eduard Gorbunov. 3PC: Three Point Compressors for Communication-Efficient Distributed Training and a Better Theory for Lazy Aggregation. In *International Conference on Machine Learning (ICML)*, 2022.
- [40] Haoyu Zhao, Boyue Li, Zhize Li, Peter Richtárik, and Yuejie Chi. BEER: Fast $O(1/T)$ Rate for Decentralized Nonconvex Optimization with Communication Compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [41] Yujun Wang, Lu Lin, and Jinghui Chen. Communication-Compressed Adaptive Gradient Method for Distributed Nonconvex Optimization. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2022.
- [42] Ron Dorfman, Shay Vargaftik, Yaniv Ben-Itzhak, and Kfir Yehuda Levy. DoCoFL: Downlink compression for cross-device federated learning. In *International Conference on Machine Learning (ICML)*, 2023.
- [43] Tao Lin, Sebastian U. Stich, Luis Barba, Daniil Dmitriev, and Martin Jaggi. Dynamic model pruning with feedback. In *International Conference on Learning Representations (ICLR)*, 2020.
- [44] Yoel Drori. *Contributions to the Complexity Analysis of Optimization Algorithms*. PhD thesis, Tel-Aviv University, 2014.
- [45] Adrien B. Taylor, Julien M. Hendrickx, and François Glineur. Exact Worst-Case Performance of First-Order Methods for Composite Convex Optimization. *SIAM Journal on Optimization*, 27(3):1283–1313, 2017.
- [46] Adrien Taylor, Bryan Van Scoy, and Laurent Lessard. Lyapunov Functions for First-Order Methods: Tight Automated Convergence Guarantees. In *International Conference on Machine Learning (ICML)*, 2018.
- [47] Manu Upadhyaya, Sebastian Banert, Adrien B. Taylor, and Pontus Giselsson. Automated tight Lyapunov analysis for first-order methods. *Mathematical Programming*, 209(1):133–170, 2025.

- [48] Baptiste Goujaud, Aymeric Dieuleveut, and Adrien Taylor. On Fundamental Proof Structures in First-Order Optimization. In *Conference on Decision and Control (CDC)*, 2023.
- [49] Alexandre d’Aspremont. Smooth Optimization with Approximate Gradient. *SIAM Journal on Optimization*, 19(3):1171–1183, 2008.
- [50] Olivier Devolder, François Glineur, and Yurii Nesterov. First-order methods of smooth convex optimization with inexact oracle. *Mathematical Programming*, 146:37–75, 2014.
- [51] Mark Schmidt, Nicolas Roux, and Francis Bach. Convergence Rates of Inexact Proximal-Gradient Methods for Convex Optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2011.
- [52] Etienne De Klerk, Francois Glineur, and Adrien B Taylor. Worst-Case Convergence Analysis of Inexact Gradient and Newton Methods Through Semidefinite Programming Performance Estimation. *SIAM Journal on Optimization*, 30(3):2053–2082, 2020.
- [53] Lukas Balles and Philipp Hennig. Dissecting Adam: The Sign, Magnitude and Variance of Stochastic Gradients. In *International Conference on Machine Learning (ICML)*, 2018.
- [54] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Animashree Anandkumar. signSGD: Compressed Optimisation for Non-Convex Problems. In *International Conference on Machine Learning (ICML)*, 2018.
- [55] Yoel Drori and Marc Teboulle. Performance of first-order methods for smooth convex minimization: a novel approach. *Mathematical Programming*, 145(1):451–482, 2014.
- [56] Adrien B. Taylor, Julien M. Hendrickx, and François Glineur. Smooth strongly convex interpolation and exact worst-case performance of first-order methods. *Mathematical Programming*, 161(1-2):307–345, 2017.
- [57] MOSEK ApS. *MOSEK Optimizer API for Python 11.0.21*, 2025. URL <https://docs.mosek.com/11.0/pythonapi/index.html>.
- [58] Baptiste Goujaud, Céline Moucer, François Glineur, Julien M. Hendrickx, Adrien B. Taylor, and Aymeric Dieuleveut. PEPit: computer-assisted worst-case analyses of first-order optimization methods in Python. *Mathematical Programming Computation*, 16(3):337–367, 2024.
- [59] Adrien B. Taylor, Julien M. Hendrickx, and François Glineur. Performance estimation toolbox (PESTO): automated worst-case analysis of first-order optimization methods. In *Conference on Decision and Control (CDC)*, 2017.
- [60] Adrien Taylor and Francis Bach. Stochastic first-order methods: non-asymptotic and computer-aided analyses via potential functions. In *Conference on Learning Theory (COLT)*, 2019.
- [61] Miles Cranmer. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. *arXiv:2305.01582 [physics]*, May 2023.
- [62] Maryam Fazel, Haitham Hindi, and Stephen P. Boyd. Log-det heuristic for matrix rank minimization with applications to Hankel and Euclidean distance matrices. In *American Control Conference (ACC)*, 2003.
- [63] Wolfram Research, Inc. Mathematica, Version 14.2. URL <https://www.wolfram.com/mathematica>. Champaign, IL, 2024.
- [64] Oran Gannot. A frequency-domain analysis of inexact gradient methods. *Mathematical Programming*, 194(1-2):975–1016, 2022.
- [65] Etienne De Klerk, François Glineur, and Adrien B Taylor. On the worst-case complexity of the gradient method with exact line search for smooth strongly convex functions. *Optimization Letters*, 11:1185–1199, 2017.
- [66] Baptiste Goujaud, Aymeric Dieuleveut, and Adrien Taylor. Counter-examples in first-order optimization: a constructive approach. *IEEE Control Systems Letters*, 2023. (See [arXiv 2303.10503](https://arxiv.org/abs/2303.10503) for complete version with appendices).